BİLGİSAYAR MÜHENDİSLİĞİNE GİRİŞ

Dr. Öğr. Üyesi İbrahim KÖK

YAZILIM MÜHENDİSLİĞİ

- Temel Kavramlar
- Yazılım Özellikleri
- Yazılım yaşam döngüsü
- Yazılım geliştirme modelleri
- Yazılımda Pratik Prensipler

Yazılım Nedir?

Yazılım: Herhangi bir boyuttaki herhangi bir tür **donanımda çalışan bilgisayar programını** ve **basılı veya elektronik ortamdaki her tür dokümanı** içeren ürünü ifade etmektedir.

- Herhangi bir donanım üzerinde çalışan,
- Tanımlanmış bir işlevi yerine getiren,
- Girdi ve çıktıları olan,

Bilgisayar programı ve kullanım/bakım kılavuzları gibi belgelerden oluşan bir üründür.

Yazılım Bileşenleri

Yazılım bileşenleri;

- Mantık (Algoritma) +
- Program (kod) +
- İnsan (kullanıcı, geliştirici) +
- Veri (planlama, çözümleme ve tasarım bilgisi)+
- Belge (system ve kullanıcı dokümanları)

Yazılım Mühendisliği (YM) Nedir?

Yazılım mühendisliği tanım olarak "karmaşık yazılım sistemlerinin belirli bir hedefe ve sisteme dayalı olarak ve **işbölümü yapılarak, belirli prensipler, yöntemler ve araçlar kullanılarak** geliştirilmesidir."

- ***Bilimsel bilginin*** bilgisayar programlarının tasarımı, pratikte uygulanması, onların geliştirilmesi, *çalıştırılması* ve *devam ettirilmesiyle aşamalarında kullanılması/işe koşulması,*
- Yazılım geliştirmek, yazılımı çalıştırmak ve devam ettirmek için **sistematik, disiplinli ve ölçülebilir yaklaşımların uygulanması** [IEEEComputerSociety,1990].

Yazılım Mühendisliği Nedir?

Yazılım mühendisliği; yazılım üretimi ile ilgili tüm durumlarla ilgilenen bir mühendislik bilim dalıdır.

- Yazılım mühendisleri;
  - İşlerinde sistematik ve organize yaklaşımlar benimsemelidirler.
  - Çözmek istedikleri probleme, geliştirme kısıtlamalarına ve mevcut kaynaklara uygun araç ve teknikleri kullanmalıdırlar.

Yazılım – Bilgisayarın ilk yılları

Geçmiş yıllar

- Oldukça küçük programlar
- Tek kişinin yazdığı programlar
- Sadece alan uzmanlarının geliştirip yine kendilerinin kullandığı programlar
- Bazı programlama dillerinde bilinen algoritmaların kullanım eğilimi

Yazılım – Günümüz

Günümüz Programlar ve yazılımları

- Oldukça büyük ve karmaşık
- Uzun süreler zarfında birbirleriyle işbirliği içinde çalışan takımlar tarafından geliştiriliyorlar.
- Geliştiriciler artık geliştirilen yazılımın son kullanıcısı değiller.
    - Sistemin asıl kullanıcılarının alanla ilgili uzmanlık bilgileri yok.

Yazılım Çeşitleri

- **Genel**
  - Pek çok farklı müşteriye satılmak üzere üretilmiş hazır ticari ürünler (Commercial Off The Shelf – COTS)

- **Ismarlama / İsteğe özelleştirilmiş**
  - Tek bir müşteri için onun belirtimleri (ihtiyaçları) doğrultusunda hazırlanmış

Yazılım Türleri

- **Sistem yazılımları;** Diğer programlara hizmet sunmak üzere hazırlanmış programlar (Derleyiciler, işletim sistemleri, vb.)
- **Uygulama yazılımları;** Kullanıcıların işlerine çözüm sağlayan programlardır.
- Mühendislik ya da bilimsel yazılımlar; Mühendislik ve bilimsel hesaplamalarda kullanılmak üzere hazırlanmış programlar. Büyük hacimli verilerle uğraşırlar.
- Gömülü yazılımlar; Donanım ile çok sıkı ilişkidedir. Denetim amaçlıdır.
- Kurumsal Yazılım: Belirli ticari iş gereksinimlerine yönelik programlar. İş süreçleri ile ilgili bilgiye sahip olmalıdır. Genellikle müşteriye özel tasarlanır.
- Yapay zeka yazılımları

Mühendislik Nedir?

- Doğadaki maddenin ve enerji kaynaklarının insanların kullanımı için yararlı hale getirilmesi için bilimsel ve matematiksel prensiplerin uygulanmasıdır**.**

- **Mühendisler**

  - Uygun olan yerlerde **teori + metot + araçları uygulayarak** işlerin yürümesini sağlarlar.
  - Çeşitli ***kısıtlamalar*** içerisinde çözümler bulmaya çalışırlar.

Mühendislik Aktivitelerinin Prensipleri

- Mühendislik aktivitelerinin prensipleri
  - Tüm projeler
    - Umulan/önceden tahmin edilen bütçe
      - MALİYET
    - Umulan/önceden tahmin edilen zaman çizelgesi
      - ZAMAN
    - Müşteri gereksinimine/isterlerine uygun ,
      - KALİTE

    şekilde tamamlanması gerekmektedir.

Maliyet + Zaman + Kalite



Yazılım Mühendisliğinin Önemi

- Gün geçtikçe yazılımla kontrol edilen sistemler artış göstermektedir.
- Tüm gelişmiş ulusların ekonomileri günümüzde yazılıma dayanmaktadır.
- Yazılım mühendisliği harcamaları ülkelerin gayri safi milli hasılalarının kayda değer bir bölümünü oluşturmaktadır.

Yazılım Mühendisliğinin Önemi

- Yazılım maliyetleri sistem maliyetlerinin büyük kısmını oluşturmaktadır.
- Bilgisayar üzerinde çalışacak yazılımın maliyeti donanımın maliyetinden genellikle daha fazladır.
- Yazılımın sürdürülebilirlik maliyeti geliştirme maliyetinden daha fazladır.
  - Uzun süreli kullanılacak sistemler için, sürdürülebilirlik maliyetleri geliştirme maliyetlerinin birkaç katı olabilir.
- Yazılım mühendisliği maliyet-etkin yazılımlar geliştirmekle ilgilidir.

Yazılım Mühendisi

- Sadece bir kodlayıcı, yani programlayıcı değildir.
- Yazılım mühendisliği disiplinini uygulayarak yazılım geliştiren kişidir.
- Herhangi bir programlama dilini bilen bir kişi programcı olabilir ama eğitimini almadan yazılım mühendisliği işini yapamaz.
  - Salt kodlayıcı değil ama kod yazma tekniklerini çok iyi bilir.
  - İyi bir belge düzenleyici olmayabilir ama çok iyi gözden geçiricidir.
  - Uygulama alanında az bilgisi olabilir fakat kullanıcı isteklerini nasıl aktarabileceğini bilir.

Yazılım Özellikleri

1. Yazılım geliştirilir (developed), *klasik* anlamda imal (manufactured) edilmez.

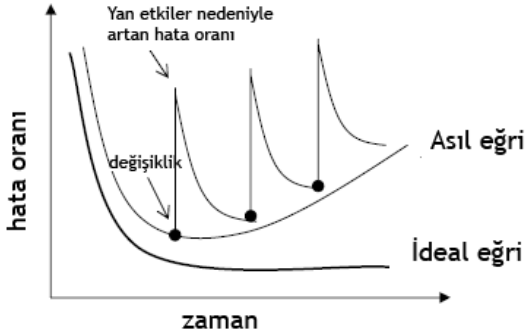


Yazılım Özellikleri

2. Yazılım aşınmaz / yıpranmaz, fakat yapılan değişiklikler nedeniyle geriler.
   - Pek çok değişiklikten sonra baştan tasarlanması gerekebilir.



DONANIM



YAZILIM

Yazılım Problemleri

- Öngörülen zamanın gerisinde kalma
- Bütçeyi aşma
- Düşük Kalite
  - Güvenilir olmayan yazılım
  - Kullanıcı taleplerinin karşılanmasında yetersizlik
  - Sürekliliğinin sağlanmasındaki zorluk

Yazılım Hakkında Yanılgılar

MÜŞTERİ AÇISINDAN;

- **Programın yazılmasına başlanması için amaçları genel olarak belirlemek yeter, ayrıntılar sonra kararlaştırılabilir. Nasıl olsa yazılım esnektir.** (Belirsiz gereksinimler, çürük atılmış temele benzer.)
- **Yazılım esnektir. Değişen gereksinimler kolayca sisteme uyarlanabilir.**

(Yazılım yaşam döngüsünde ilerledikçe, değişen gereksinimleri yazılıma uyarlamanın bedeli üstel olarak artar.)

- **Sonuç: Yazılım esnek bir oyun hamurundan çok kil veya cam gibidir.**

(Çevik süreçlerle esnekliğin arttırılması hedeflenmektedir.)

Yazılım Hakkında Yanılgılar

PROGRAMCI AÇISINDAN;

- **Yazılımı tamamlayıp müşteriye teslim edince işimiz biter.** (Yazılım üstünde harcanan çabanın yarısından fazlası, yazılımın müşteriye ilk teslimatından sonra harcanmaktadır.)
- **Yazılımı tamamlamadan kalitesini ölçemem.** (Kalite güvence yöntemleri yazılım hayat döngüsünün her aşamasında uygulanabilir.)
- **Yazılım eşittir program** (Dokümantasyon ve sınama çalışmaları?)
- **Yazılım mühendisliğinin gereklerini uygulayarak boşuna çaba harcıyoruz.** (Haritası olmayan yolunu kaybeder)

Yazılım Hakkında Yanılgılar

İDARİ AÇIDAN;

- **İşler yetişmiyorsa takıma yeni programcılar ekleriz.**

  (Yazılım hayat döngüsü içerisinde ilerledikçe, yeni elemanların yazılıma hakim olması üstel olarak zorlaşır.)

- **Geliştirmesini üstlendiğim yazılımı tamamen veya kısmen fason yaptırırım.**

  (Dışarıya verilen işin takibi zor veya imkansız olacaktır)

- **Açık kaynak yazılım üretirsem kar edemem**.
  (Danışmanlık hizmetleri ile kar edilebilir.)

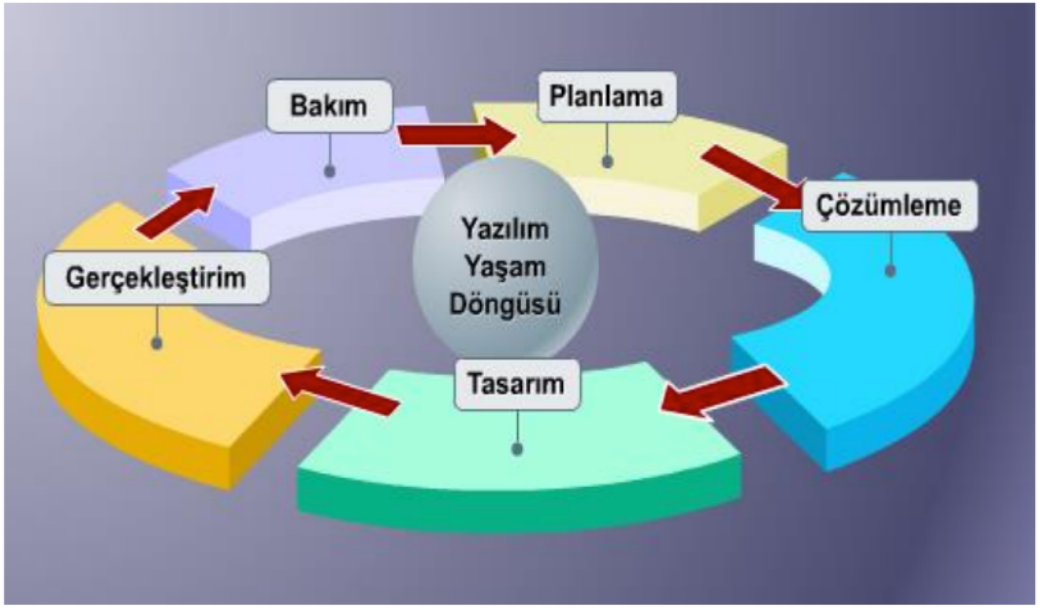
İyi Yazılımın Özellikleri

- Yazılım **kullanıcısına istenen işlevleri ve performansı sunmalı** ve buna ek olarak:
  - Sürdürülebilirlik (maintainable)
    - Yazılım değişen ihtiyaçlara göre gelişebilmelidir.
  - Güvenilebilirlik (dependable)
    - Yazılım güvenilir olmalıdır
  - Verimlilik (efficiency)
    - Yazılım sistem kaynaklarını boşa harcamamalıdır.
  - Kullanılabilirlik (usability)
    - Sistem tasarlandığı kullanıcıları için kolay kullanılabilir olmalıdır.

Yazılım Yaşam Döngüsü

Herhangi bir yazılımın, üretim aşaması ve kullanım aşaması da dahil olmak üzere geçirdiği tüm aşamalar biçiminde tanımlanır. Bu aşamalara **çekirdek süreçler** de denilmektedir.

Planlama, Çözümleme/analiz, Tasarım, Gerçekleştirim, Bakım

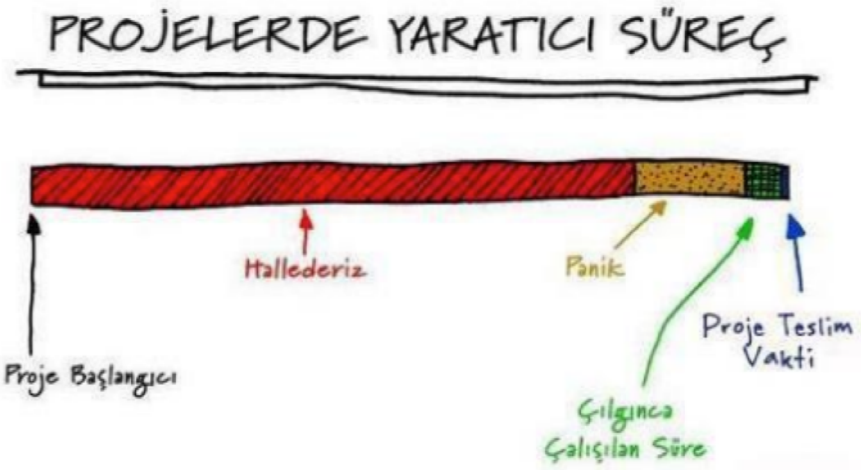


Yazılım Yaşam Döngüsü

- **Planlama:** Personel ve donanım **gereksinimlerinin çıkarıldığı**, fizibilite (olurluk-yapılabilirlik) çalışmasının yapıldığı ve proje planının oluşturulduğu aşamadır.
- **Çözümleme/Analiz:** Sistem gereksinimlerinin ve işlevlerinin ayrıntılı olarak çıkarıldığı aşama. Mevcut var olan işler incelenir, temel sorunlar ortaya çıkarılır.
- **Tasarım:** Belirlenen gereksinimlere yanıt verecek yazılım sisteminin temel yapısının oluşturulduğu aşamadır.
  - Mantıksal; önerilen sistemin yapısı anlatılır, olası örgütsel değişiklikler önerilir.
  - Fiziksel; yazılımı içeren bileşenler ve bunların ayrıntıları.
- **Gerçekleştirim:** Kodlama, test etme ve kurulum çalışmalarının yapıldığı aşamadır.
- **Bakım:** Hata giderme ve yeni eklentiler yapma aşaması (teslimden sonra).

Yazılım Yaşam Döngüsü Ne sağlar ?

- Mühendislik faaliyetlerinin (üretim, işletme ve bakım aşamaları) yazılıma uyarlanması
- Üretimden kullanıma kadar tüm sürecin fazlara ayrılması ve böylece yönetim kolaylığı
- Her bir aşamada ne yapılacağı bilgisi
- Her bir aşamadaki standartlara uyumluluk ile kaliteli yazılım geliştirme vb.



Yazılım Süreci

- Görevler arasındaki genel akış ve bağımlılık seviyeleri
  - Problemi anlama (iletişim ve analiz)
  - Çözüm planlama (yazılım tasarımı)
  - Planı yürütme (kod üretme)
  - Doğruluk için sonuçları inceleme (test ve kalite güvencesi)

Yazılım Geliştirme Modelleri

- Kodla Düzelt
- Çağlayan (Şelale) Modeli
- V Modeli
- Helezonik/Spiral Model
- Prototipleme Modeli
- Evrimsel Model
- Diğer (artımlı, agile vb.)

Yazılım Süreç Modelleri

- **Kodla ve Düzelt**

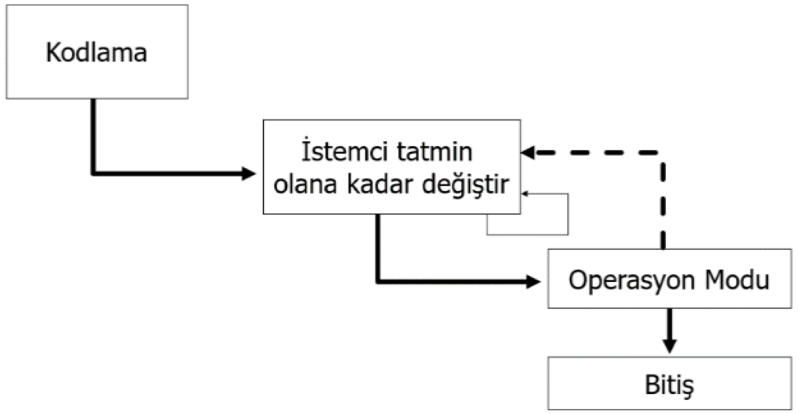
Genellikle resmi olmayan bir ürün fikriyle başlar ve **program ürün "hazır" olana kadar** ya da gerekli zaman bitene kadar kodlama yapılarak devam eder.

Avantajları

- Herhangi bir planlamaya ihtiyaç duyulmaz
- Çok küçük projelerde ya da kısa ömürlü prototiplerde uygulanabilir
- Program aşamaları çabuk geçilir
- Uzman görüşüne ihtiyaç düşüktür herkes bu modeli kullanabilir

Dezavantajları

- Kontrollü değildir.
- Kaynak planlaması yoktur
- Bitiş süresi belli değildir
- Hataların bulunması ve doğrulaması zordur
- Kodları düzeltmek maliyetli olabilir



Yazılım Süreç Modelleri

- **Çağlayan/Şelale Modeli**



Yaşam döngüsü temel adımları baştan sona en az bir kez izleyerek gerçekleştirilir
İyi tanımlı projeler ve üretimi az zaman gerektiren yazılım projeleri için uygun bir modeldir.

Yazılım tanımlamada belirsizlik yok (ya da az) ise ve yazılım üretimi çok zaman almayacak ise uygun bir süreç modelidir.

Avantajları

+ Kullanımı ve anlaması basittir, Yönetimi kolaydır
+ Projenin safhaları ayrı olduğundan iş bölümü ve iş planı projenin en başında net bir şekilde bellidir. Bu durum projenin yönetimini de oldukça kolay hale getirir.
+ Şelale Modeli çok küçük ve gereksinimleri çok iyi anlaşılmış projelerde iyi çalışır.

Dezavantajları

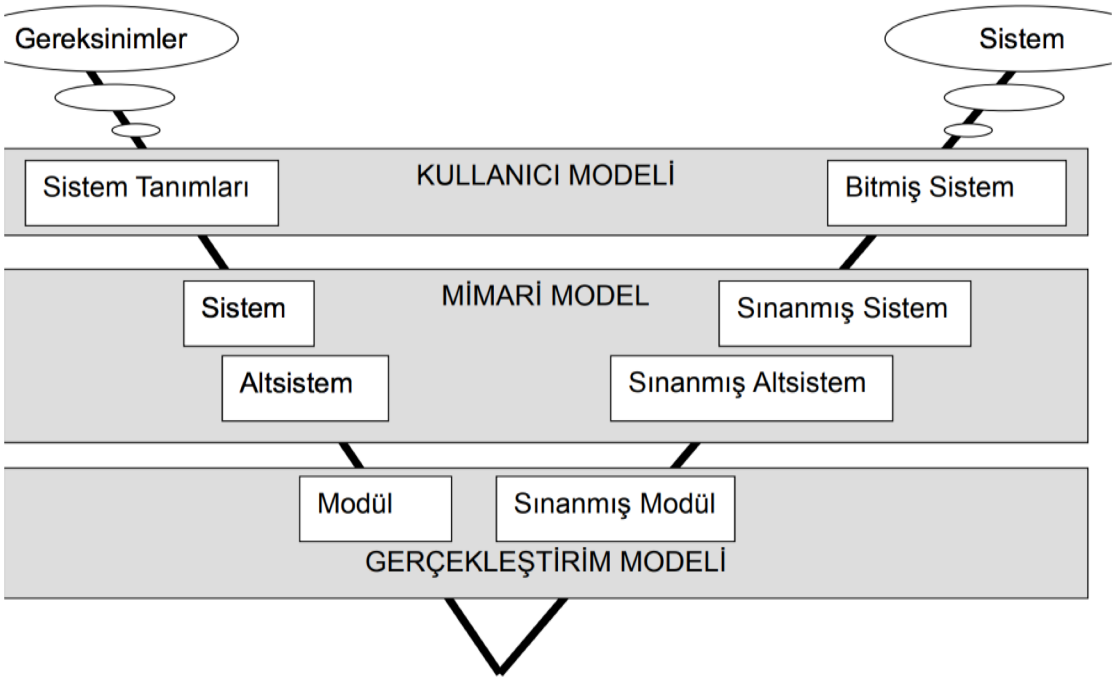
- Karmaşık ve nesne yönelimli projeler için uygun değildir
- Devam eden ve uzun projeler için zayıftır
- Müşteri memnuniyetini sağlamak çok zordur çünkü gelişim ve değişime açık bir model **değildir**.

Yazılım Süreç Modelleri

V Modeli

Doğrusal bir yönde ilerlemek yerine, süreç adımları kodlama evresinden sonra yukarıya doğru eğim alır ve tipik V şeklini oluşturur. **Belirsizliklerin az, iş tanımlarının belirgin olduğu BT projeleri için uygun** bir modeldir.

Sol taraf üretim, sağ taraf sınama/test işlemleridir.
V süreç modelinin temel çıktıları; Kullanıcı modeli, Mimari model, Gerçekleştirim modeli



V Modeli

V süreç modelinin temel çıktıları;

Kullanıcı Modeli; Geliştirme sürecinin kullanıcı ile olan ilişkileri tanımlanmakta ve sistemin nasıl kabul edileceğine ilişkin sınama belirtimleri ve planları ortaya çıkarılmaktadır.

Mimari Model; Sistem tasarımı ve oluşacak alt sistem ile tüm sistemin sınama işlemlerine ilişkin işlevler.

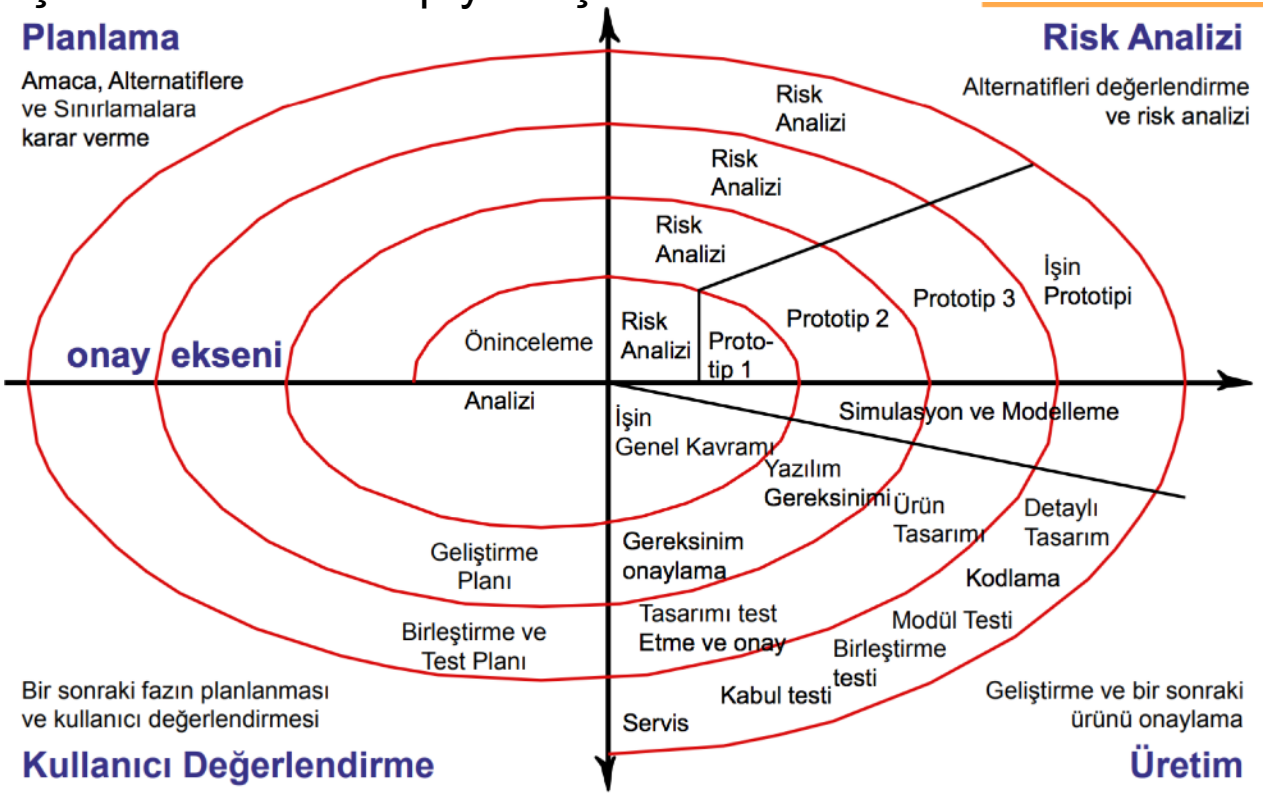
Gerçekleştirim Modeli; Yazılım modüllerinin kodlanması ve sınanmasına ilişkin fonksiyonlar.

Avantaj / Dezavantaj

+ Proje yönetimi tarafında takibi kolaydır
+ Kullanımı kolaydır.
- Aynı zamanda gerçekleştirilebilecek olaylara kolay imkan tanımaz.
- Aşamalar arasında tekrarlamaları kullanmaz.
- Risk çözümleme ile ilgili aktiviteleri içermez.

Helezonik/Spiral/Sarmal Model

Hem klasik çevrim hem de prototipleme yöntemlerinin iyi yönlerinin birleştirilmesiyle oluşturulmuştur. Doğrudan tanımlama, tasarım,... vs gibi bir faz yoktur. Yinelemeli artımsal bir yaklaşım vardır. Prototip yaklaşımı vardır. Risk Analizi Olgusu ön plana çıkmıştır.



Yazılım Geliştirme Modelleri

Helezonik/Spiral/Sarmal Model

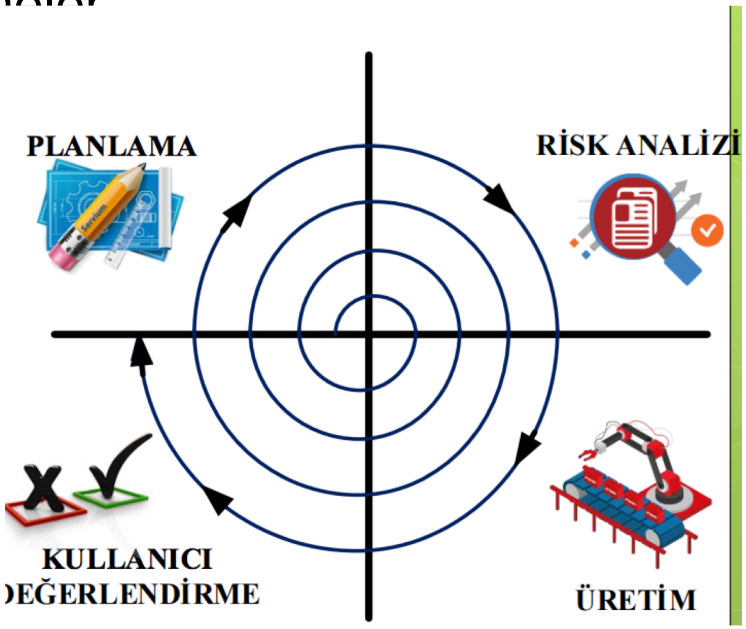
1. Planlama: Üretilecek ara ürün için planlama, amaç belirleme, bir önceki adımda üretilen ara ürün ile bütünleştirme

2. Risk Analizi: Risk seçeneklerinin araştırılması ve risklerin belirlenmesi

3. Üretim: Ara ürünün üretilmesi

4. Kullanıcı Değerlendirmesi: Ara ürün ile ilgili olarak kullanıcı tarafından yapılan sınama ve değerlendirmeler

Kullanıcı katkısı; Üretim süreci boyunca ara ürün üretme ve üretilen ara ürünün kullanıcı tarafından sınanması temeline dayanır. Yazılımı kullanacak personelin sürece erken katılması ileride oluşabilecek istenmeyen durumları engeller.

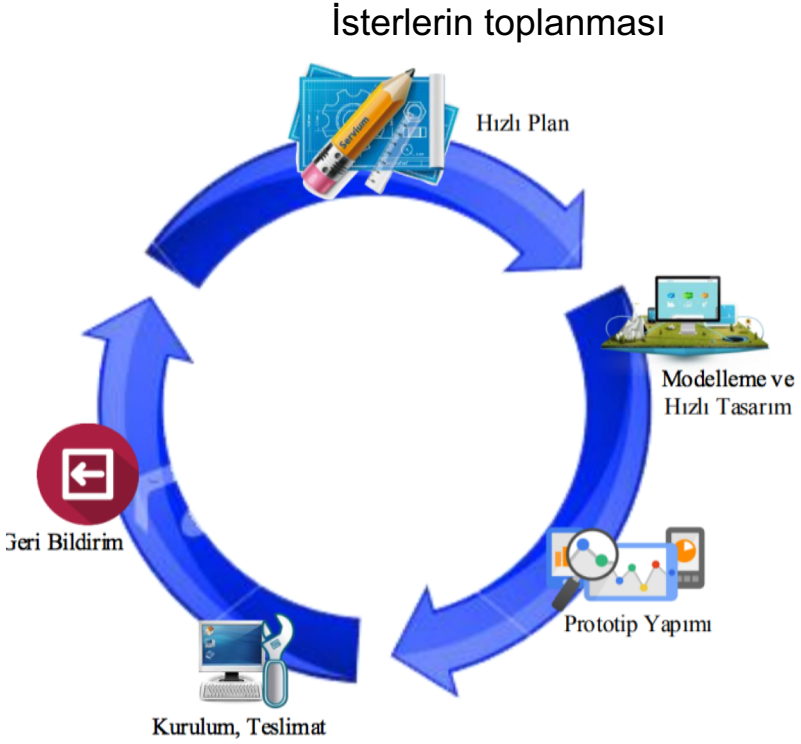


Yazılım Süreç Modelleri

Prototipleme

Müşteri yazılım ürününden ne beklediğini belirtir, ancak ayrıntılı giriş, işleme ve çıkış işlemleri tanımlayamadığı durumlarda kullanabilecek bir yöntemdir.

- **Belirsizliklerin bulunduğu durumlarda prototip (örnek, ön ürün) yaklaşımı en iyi yazılım geliştirme yöntemi olabilir.**
- Gereksinim tanımlama fazında hızlıca yapılan kısmi gerçekleştirme.
- Gereksinimler netleştikçe prototip kullanıcı ihtiyaçlarına göre düzeltilir
- Müşteri memnun olana kadar düzeltmelere devam et



Prototipleme

Avantaj /dezavantaj

+ Kullanıcı sistem gereksinimlerini görebilir
+ Karmaşa ve yanlış anlaşılmaları engeller
+ Yeni ve beklenmeyen gereksinimler netleştirilebilir
+ Risk kontrolü sağlanır

- Kaynak maliyetlerinin kestirim zorluğu
- Bir prototip çalışmasının ne kadar süreceği net olmayabilir,
- Bir prototipin ne kadar iş yükü gerektireceği kolayca kestirilememekte ve dolayısıyla yönetimi zorlaşmaktadır.

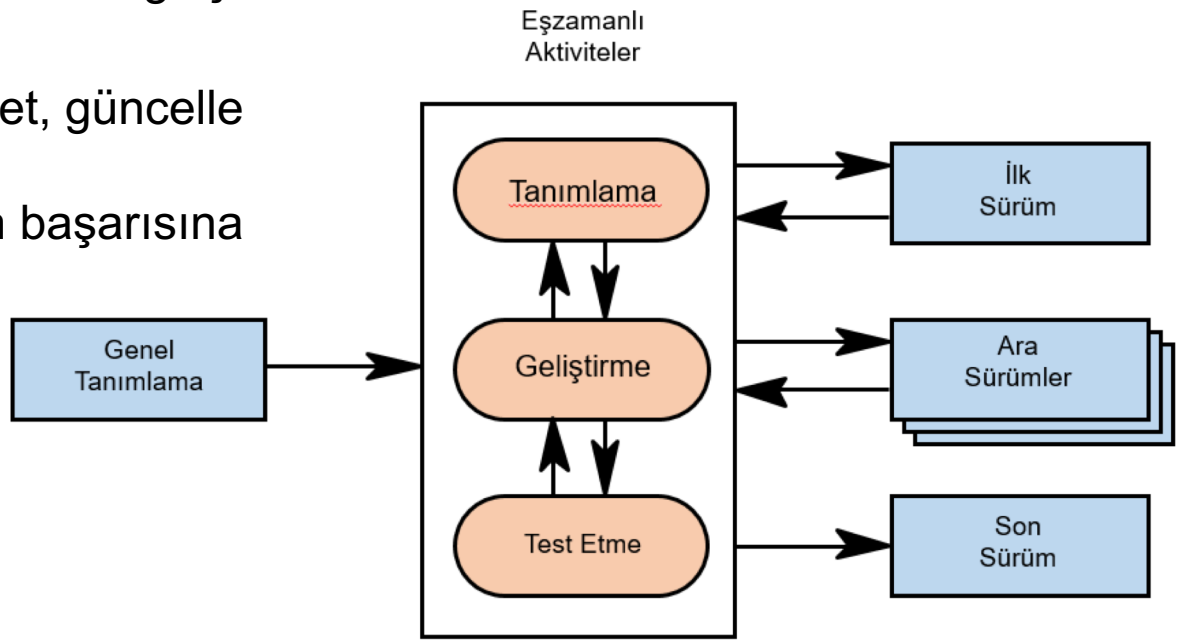


Yazılım Süreç Modelleri

- **Evrimsel Geliştirme Modeli**

Daha çok coğrafik olarak geniş alana yayılmış, çok birimli organizasyonlar (banka uygulamaları) için önerilmektedir. Her aşamada üretilen ürünler, üretildikleri alan için tam işlevselliği içermektedirler.

Pilot uygulama kullan, test et, güncelle diğer birimlere taşı.
Modelin başarısı ilk evrimin başarısına bağımlıdır.



Yazılımda Pratik Prensipler

- Keep it simple stupid (KISS).
- Açık bir vizyon herhangi bir yazılım projesi için gereklidir.
- Her zaman başka birinin işlerini ona göre yürüteceğinizi düşünerek tanımla, tasarla ve uygula.
- Gelecekteki değişikliklere açık ol.
- Önceden yeniden kullanım için planlamak maliyeti düşürür ve yeniden kullanılan bileşenlerin ve onu gerektiren sistemin değerini artırır.

Yazılımda Pratik Prensipler

